Software Engineering

BCSE301L

Module 6

Dr. Naveenkumar Jayakumar
Dept. of Computational Intelligence
PRP-217-4

Software Maintenance

- The general process of changing a system after it has been delivered
- The process of modifying and updating a software system after it has been delivered to the customer.
- It is necessary to ensure that the software continues to meet the needs of the users over time

Software Maintenance - Goals

- To keep the software system working correctly, efficiently, and securely.
- To ensure that it continues to meet the needs of the users which includes
  - Fixing bugs
  - Adding new features
  - Improving performance
  - Updating the software to work with new hardware or software systems.

Software Maintenance – Three Types

Fault repairs to fix bugs and vulnerabilities.

Environmental adaptation to adapt the software to new platforms and environments.

Functionality addition to add new features and to support new requirements.

Software Maintenance – Three Types

- Fault repairs to fix bugs and vulnerabilities.
  - Coding errors are usually relatively cheap to correct;
  - Design errors are more expensive because they may involve rewriting several program components.
  - Requirements errors are the most expensive to repair because extensive system redesign may be necessary.

Software Maintenance – Three Types

- Environmental adaptation to adapt the software to new platforms and environments.
  - This type of maintenance is required when some aspect of a system's environment, such as the hardware, the platform operating system, or other sup- port software, changes.
  - Application systems may have to be modified to cope with these environmental changes.

Software Maintenance – Three Types

- Functionality addition to add new features and to support new requirements.
  - This type of maintenance is necessary when system requirements change in response to organizational or business change.
  - The scale of the changes required to the software is often much greater than for the other types of maintenance.

Software Maintenance –Types

- Imagine you have developed a mobile app called "TaskMaster," which helps users manage their daily tasks. Over time,
  1. Users report that when they try to delete a task in TaskMaster, the app crashes
  2. A new version of the Android operating system is released, and TaskMaster no longer displays notifications properly on devices running this version.
  3. Users request a feature to categorize tasks into different folders (e.g., Work, Personal, Shopping). They also suggest adding a dark mode for better visibility at night.
  4. The development team notices that the codebase for TaskMaster is becoming difficult to maintain due to outdated libraries and poor documentation.

Software Maintenance –Types

- For 1.
  - Fixing bugs, errors, or defects in the software.
  - The development team investigates the issue, identifies the root cause (e.g., a null pointer exception), and releases a patch to fix the crash.
- For 2.
  - Modifying the software to work in a new environment or with new technologies.
  - The team updates the app to ensure compatibility with the latest OS version, such as modifying notification APIs or adjusting UI elements.

Software Maintenance –Types

- For 3.
  - Improving the software's performance, usability, or functionality based on user feedback..
  - The team implements these enhancements, making the app more user-friendly and efficient.
- For 4.
  - Making changes to prevent future problems and improve maintainability.
  - They refactor the code, update dependencies, and write better documentation to ensure the app remains easy to modify and extend in the future.

Software Maintenance – Types

Corrective maintenance	• It is universally used to refer to maintenance for fault repair.
Adaptive maintenance	• sometimes means adapting to a new environment and sometimes means adapting the software to new requirements.
Perfective maintenance	• sometimes means perfecting the software by implementing new requirements; in other cases, it means maintaining the functionality of the system but improving its structure and its performance.
Preventative Maintenance	• making necessary changes, upgrades, adaptations and more to prevent future problems

Software Maintenance – Types

- **Corrective Maintenance**
  - This involves fixing errors and bugs in the software system.
  - It is necessary when something goes wrong in a piece of software including faults and errors.
  - These can have a widespread impact on the functionality of the software in general and therefore must be addressed as quickly as possible.

Software Maintenance – Types

- **Preventative Maintenance:**
  - This involves making necessary changes, upgrades, adaptations and more to prevent future problems.
  - Preventative software maintenance may address small issues which at the given time may lack significance but may turn into larger problems in the future.
  - These are called latent faults which need to be detected and corrected to make sure that they won't turn into effective faults.

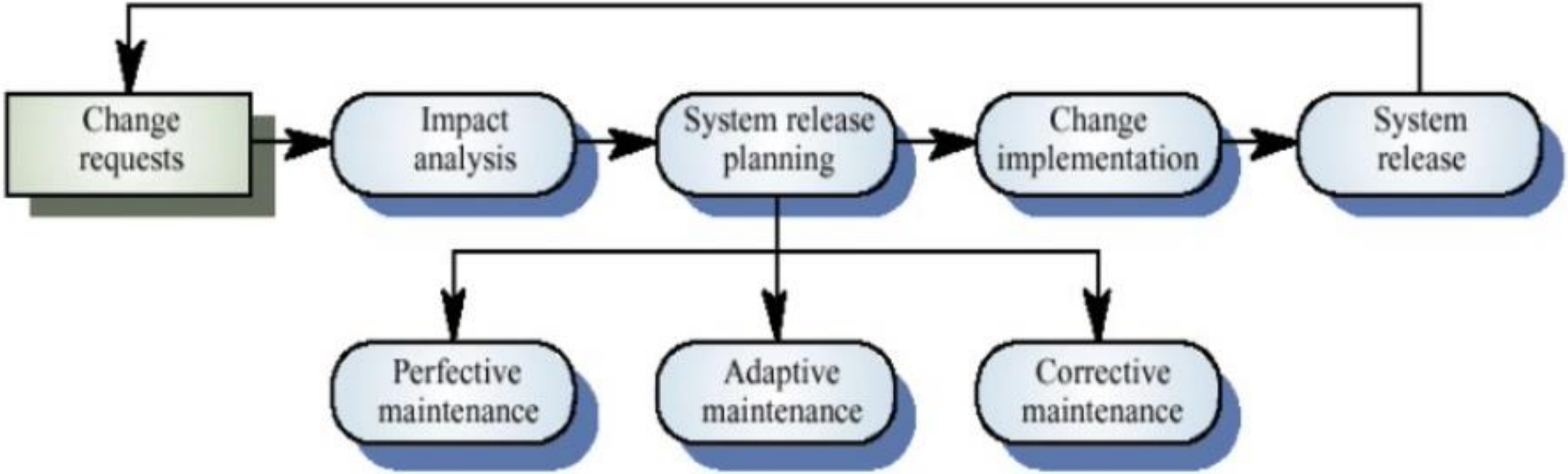
Software Maintenance – Types

- **Perfective Maintenance**
  - This involves improving functionality, performance, and reliability, and restructuring the software system to improve changeability.
  - It is done to boost performance and improve the software overall.

Software Maintenance – Types

- **Adaptive Maintenance**
  - This involves modifying the software system to adapt it to changes in the environment, such as changes in hardware or software, government policies, and business rules.
  - It is done to keep up with customer needs and market changes and demands

Software Maintenance – Process



Software Configuration Management

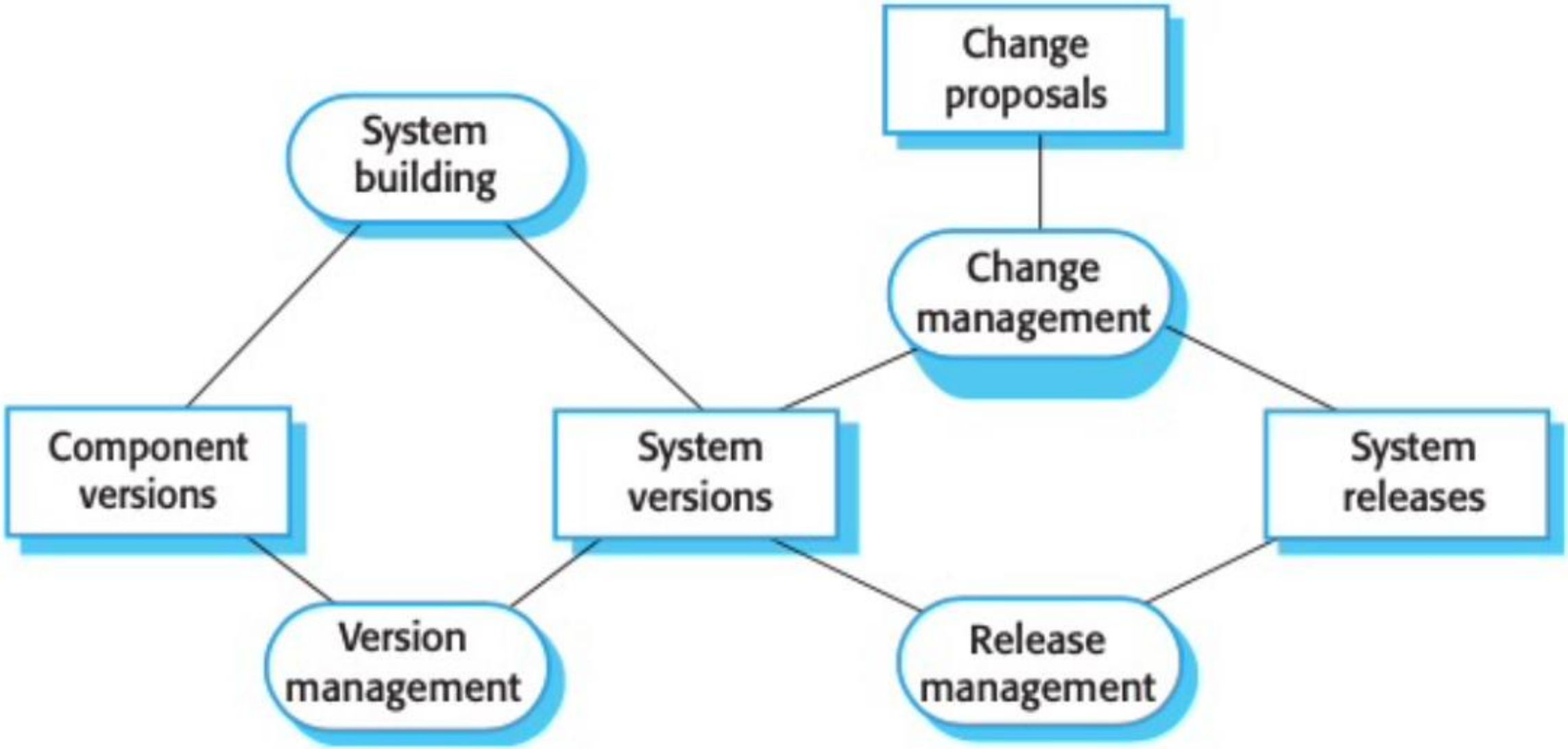
- It is essentially a set of practices used to track and control changes made to software during its development lifecycle.
- It's like keeping a detailed log of everything that goes into the software:
  - Who made the changes,
  - Why they were made, and
  - How they affect the overall program.

Software Configuration Management

- It is a systematic process used in software engineering to manage, organize, and control changes in the documents, codes, and other entities during the Software Development Life Cycle.

- The primary goal of SCM is to increase productivity with minimal mistakes.

Software Configuration Management – Activities & Tools



Software Configuration Management – Activities & Tools

The configuration management of a software system product involves four closely related activities:

Version control	System building	Change management	Release management

Software Configuration Management – Activities & Tools

Version control	• This involves keeping track of the multiple versions of system components and ensuring that changes made to components by different developers do not interfere with each other.
System building	• This is the process of assembling program components, data, and libraries, then compiling and linking these to create an executable system.
Change management	• This involves keeping track of requests for changes to delivered software from customers and developers, working out the costs and impact of making these changes, and deciding if the changes should be implemented.
Release management	• This involves preparing software for external release and keeping track of the system versions that have been released for customer use.

Software Configuration Management – Activities & Tools

Version Control

- Track changes to source code and other artifacts over time.
- Enable collaboration by allowing multiple developers to work on the same project simultaneously without conflicts.

Change Management

- Manage requests for changes, track their status, and ensure proper approval workflows.
- Maintain a history of all changes made to the software.

Build Automation

- Automate the process of compiling code, running tests, and packaging software for deployment.

Release Management

- Manage the release of software versions to different environments (e.g., development, testing, production).
- Ensure that the correct version of the software is deployed.

Software Configuration Management – Activities & Tools

Configuration Auditing

- Verify that the software meets its requirements and adheres to defined standards.
- Provide traceability of changes and configurations.

Baseline Management

- Establish and manage baselines (snapshots of the system at specific points in time) for reference and rollback purposes.

Collaboration and Workflow

- Facilitate communication and coordination among team members.
- Support branching and merging strategies for parallel development.

Software Reengineering

- It is the process of analyzing, redesigning, and modifying existing software systems to improve their quality, performance, maintainability, or functionality.

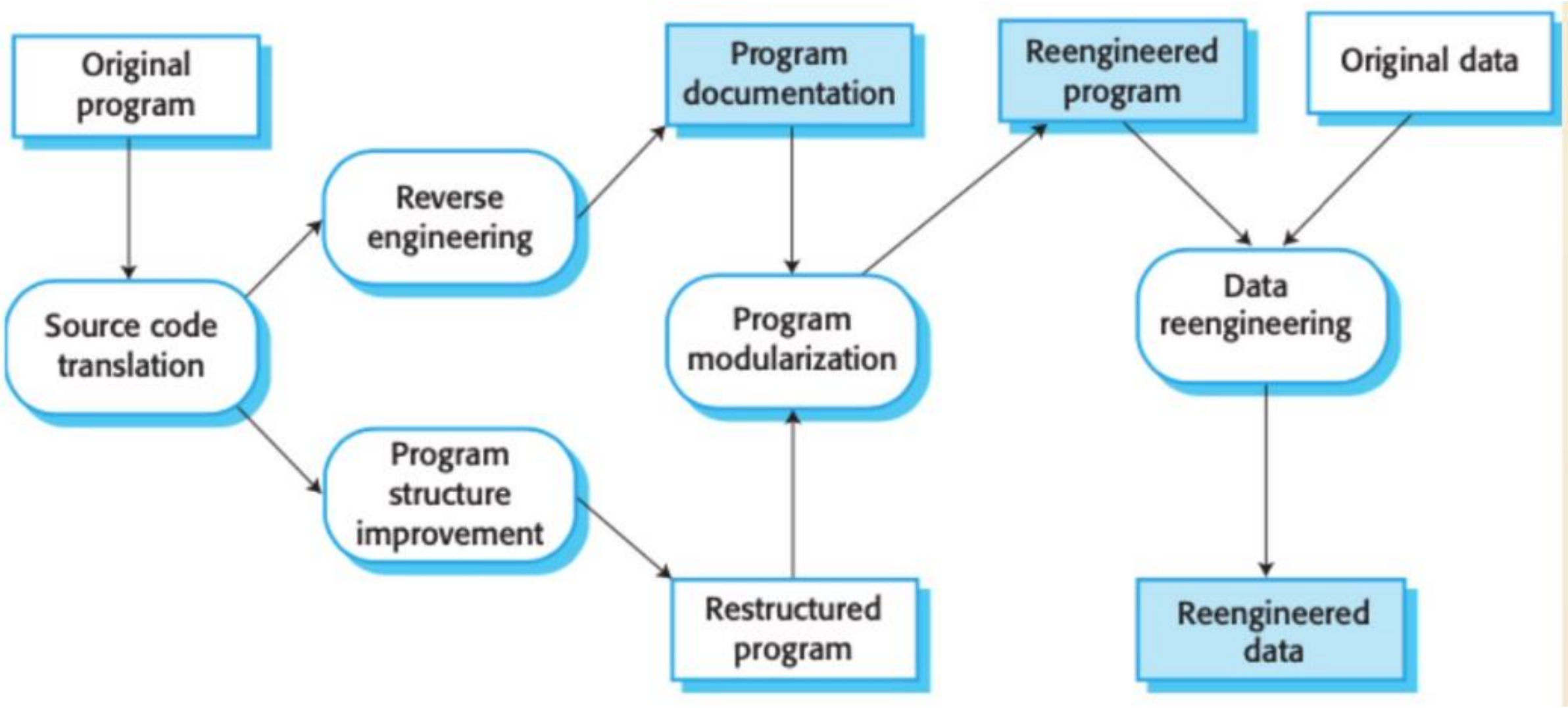
- It's essentially giving old software a makeover to make it more efficient, effective, and up-to-date

Software ReEngineering - Goals

- To improve the quality and maintainability of the software system while

- Minimizing the risks and costs associated with the redevelopment of the system from scratch.

Software ReEngineering - Process



Software Reengineering – Process - Case

- A company has an old payroll system written in COBOL that needs to be upgraded to a modern programming language (Java) with improved structure and updated data handling.

Software Reengineering – Process - Case

Original Program:

The legacy COBOL payroll system is the starting point.

Source Code Translation:

Automatically convert COBOL code to Java code using translation tools.

Example: COBOL's salary calculation logic is transformed into equivalent Java functions.

Software Reengineering – Process - Case

Reverse Engineering:	Analyze and extract high-level details (e.g., data flow, business rules).	Create a system documentation detailing how payroll is calculated and stored.
Program Structure Improvement:	Refactor the Java code by improving the control flow and removing redundant code.	Example: Simplify complex nested if statements to improve maintainability.

Software Reengineering – Process - Case

Program Modularization:

Split the payroll system into independent modules (e.g., tax calculation, employee management).

Example: Create separate Java classes for "Employee," "Tax," and "Payroll."

Data Reengineering:

Update the old flat-file storage system to a relational database (MySQL).

Example: Migrate employee records from COBOL text files to a MySQL database.

Software Reengineering – Process - Case

Reengineered Program and Data:

Deliver the final Java-based payroll system integrated with modern databases.

Example: The new system allows real-time salary calculation and integrates seamlessly with HR systems.

Software ReEngineering - Process

Source code translation

- Using a translation tool, you can convert the program from an old programming language to a more modern version of the same language or to a different language.

Reverse Engineering

- The program is analyzed, and information extracted from it. This helps to document its organization and functionality. Again, this process is usually completely automated.

Program structure improvement

- The control structure of the program is analyzed and modified to make it easier to read and understand. This can be partially automated, but some manual intervention is usually required.

Software ReEngineering - Process

Program modularization

- Related parts of the program are grouped together, and, where appropriate, redundancy is removed. In some cases, this stage may involve architectural refactoring (e.g., a system that uses several different data stores may be refactored to use a single repository). This is a manual process.

Data reengineering

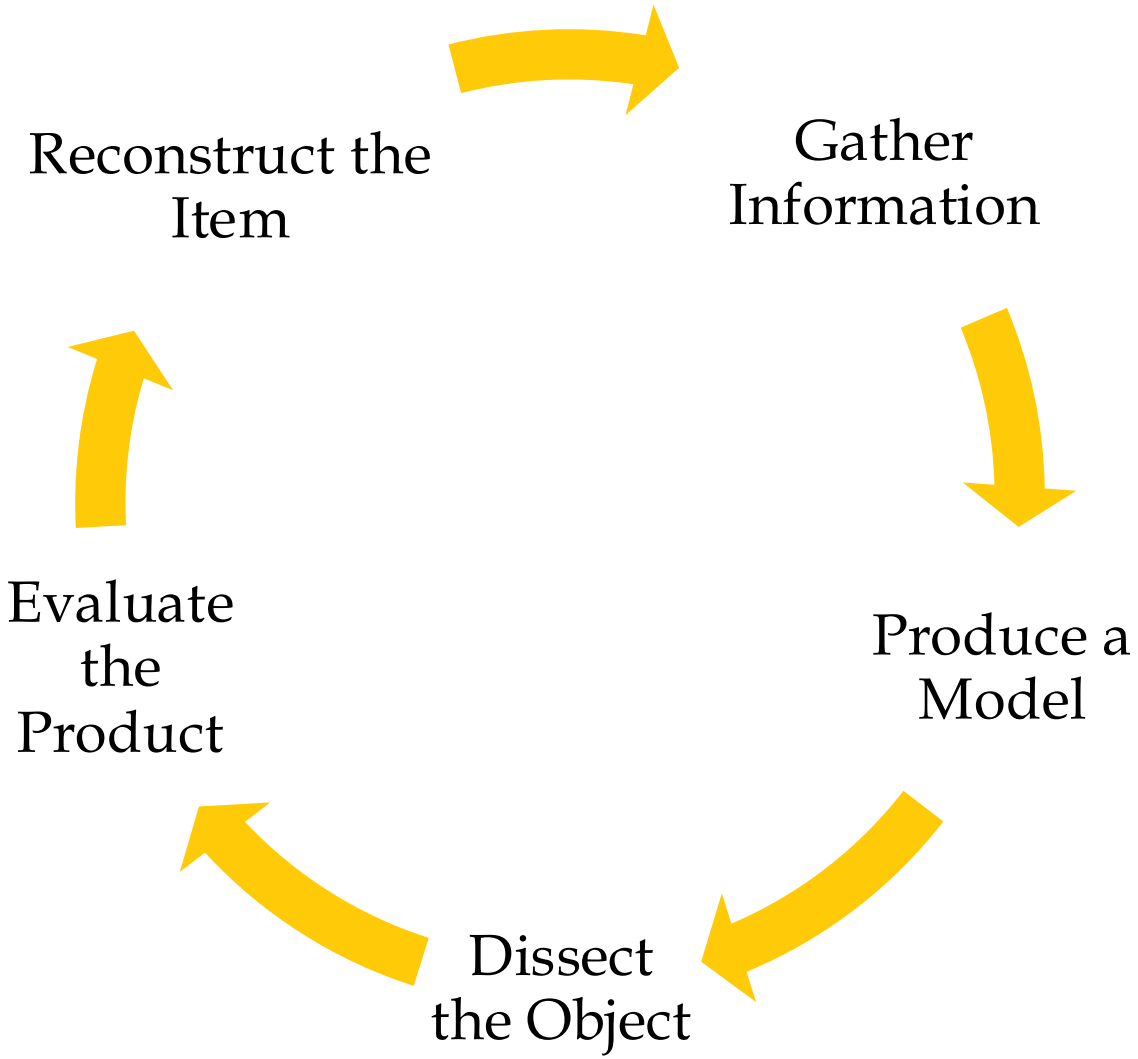
- The data processed by the program is changed to reflect program changes. This may mean redefining database schemas and converting existing databases to the new structure. You should usually also clean up the data. This involves finding and correcting mistakes, removing duplicate records, and so on. This can be a very expensive and prolonged process.

Reverse Engineering

- The process of deconstructing a system or object to understand its design, functionality, and inner workings.

- It involves dismantling a product to unveil its internal mechanisms so engineers and other professionals can understand how it works.

Reverse Engineering - Process



Reverse Engineering - Process

- **Gather information:**
  - Gathering information about the product is critical before you start the dismantling process. This process involves measuring product dimensions, identifying source designs and deducing the original coding of the device. This first step is important, so you understand the product or system before taking it apart
- **Produce a model:**
  - Based on the information collected in the previous step, you can produce a sketch or a model of the product or part. This model (If hardware embedded) is usually three-dimensional and provides an accurate representation using computer-aided design (CAD). Examining the model may help you gain information about the product's design purpose.

Reverse Engineering - Process

- **Dissect the object:**
  - The next step is to conduct a systematic disassembly of the product. Typically, you dissect the object layer by layer, following a predetermined process. You can organize the parts as you remove them to make it easier to reassemble them at a later stage. Dismantling the object sequentially may also assist you to inspect, measure and assess the individual parts to evaluate their functionalities.
- **Evaluate the product:**
  - After separating the components, you can evaluate each part to diagnose specific issues and resolve errors. The information generated from these product evaluations may help you achieve your design goals. You can record your observations and document your findings to recommend improvements. You can also scan the product sketch to a computer to help when rebuilding.

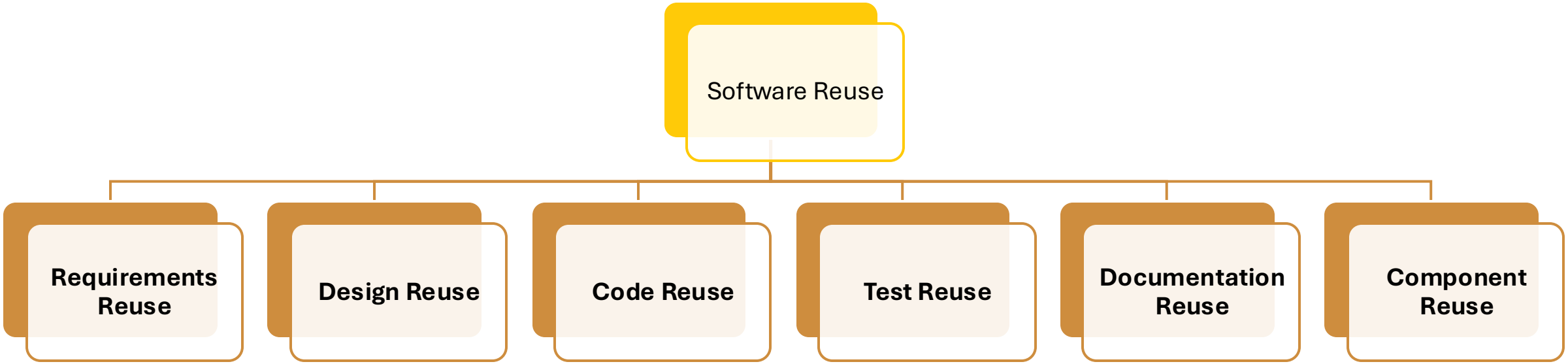
Reverse Engineering - Process

- **Reconstruct the item:**
  - During the rebuilding stage, you may make changes and test if they improve the product. This process may enable you to generate an improved version of the original. If the goal of the process is to reconstruct the object, you can combine the components to recreate the product.

Software Reuse

- Software reuse refers to the practice of using existing software components, artifacts, or knowledge in the development of new software systems.
- Instead of building every part of a system from scratch, developers leverage previously created and tested elements to save time, reduce costs, and improve quality.
- This concept is central to modern software engineering practices, as it promotes efficiency, consistency, and scalability.

Software Reuse



Software Reuse

- **Code Reuse :**
  - The most common form of reuse involves reusing source code, libraries, or modules.
  - Examples include:
    - Using open-source libraries (e.g., React for front-end development).
    - Reusing utility functions or classes across multiple projects.
- **Design Reuse :**
  - Reusing architectural patterns, design templates, or frameworks.
  - Examples include:
    - Applying the Model-View-Controller (MVC) pattern.
    - Using microservices architecture templates.

Software Reuse

- **Requirement Reuse**
  - Reusing documented requirements or specifications from previous projects.
  - This ensures consistency and reduces the effort needed to define requirements for similar systems.
- **Test Reuse**
  - Reusing test cases, test scripts, or testing frameworks.
  - For example, leveraging automated test suites for regression testing.
- **Documentation Reuse**
  - Reusing user manuals, technical documentation, or guidelines from prior projects.
  - This can significantly reduce the effort required to create new documentation.